

APPENDIX / LIVE DEMO

Proposed Work: Code Walkthrough

The four loop stages in source, and the commands that make the cluster heal itself on stage.

`tcpmon.c`

`partition.go`

`controller`

`run-demo.sh`

Kept separate from the main deck – present only if there is time for the demo.

Where each loop stage lives

Stage	Path	Language	Role
WATCH	internal/ebpf/bpf/tcpmon.c	C (CO-RE)	Per-flow smoothed RTT and byte counts
WATCH	cmd/nodeagent	Go	Loads the BPF object, merges GPU state, serves telemetry
DECIDE	internal/partition/partition.go	Go	Linear-partition DP, Evaluate, hysteresis Decider
ACT	cmd/controller	Go	Aggregates telemetry, writes the CRD, pushes assignments
HEAL	internal/controller	Go	3 s heartbeat watchdog forcing repartition across survivors
SERVE	worker/	Python	Stateless shard workers (sim, gpt2) and the router
EVAL	bench/run.py, bench/dpsim	Python, Go	Cluster ablation harness and partitioner-level evaluation

Generated gRPC stubs and the compiled BPF object are committed, so a demo needs neither protoc nor clang.

1. WATCH

Measuring latency inside the kernel

internal/ebpf/bpf/tcpmon.c

```
struct flow_val {
    __u64 bytes; // cumulative payload bytes
    __u64 srtt_us; // most recent smoothed RTT
    __u64 srtt_samples; // cumulative tcp_probe hits
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, struct flow_key);
    __type(value, struct flow_val);
} flows SEC(".maps");

SEC("tp_btf/tcp_probe")
int BPF_PROG(tcp_probe_hook, struct sock *sk,
             struct sk_buff *skb)
{
    struct tcp_sock *tp = (struct tcp_sock *)sk;
    __u32 srtt = BPF_CORE_READ(tp, srtt_us) >> 3;
    val->srtt_us = srtt;
    __sync_fetch_and_add(&val->srtt_samples, 1);
    return 0;
}

SEC("fentry/tcp_sendmsg")
int BPF_PROG(tcp_sendmsg_hook, struct sock *sk,
             struct msghdr *msg, size_t size)
```

Demo: `kubectl -n kubeedgeinfer logs -l app=keinfer-nodeagent`

• Why the kernel's own number

`srtt_us` is the smoothed RTT the TCP stack already maintains for congestion control. Reading it costs nothing, and it cannot disagree with reality the way an application-level timer can.

• CO-RE, not a rebuild

`BPF_CORE_READ` relocates struct offsets against the host BTF at load time, so one committed object file loads on any kernel with `/sys/kernel/btf/vmlinux`.

• Scoped, not global

Flows are filtered to worker ports; the controller attributes each flow to a pod by source IP.

2. DECIDE

The exact partitioner

internal/partition/partition.go

```
// stageCost: receive + compute for one stage.
// An empty stage is skipped by the router entirely,
// so it costs nothing - not even its hop.
func stageCost(in Input, worker, layers int) float64 {
    if layers == 0 {
        return 0
    }
    cost := float64(layers) * in.PerLayerMs /
        math.Max(in.Workers[worker].Speed, 0.01)
    if worker > 0 {
        cost += in.LinkMs[worker-1]
    }
    return cost
}

// Optimal solves the linear partition in  $O(N^2 * K)$ .
for w := 1; w <= k; w++ {
    for j := 0; j <= n; j++ {
        for split := 0; split <= j; split++ {
            cost := math.Max(f[split][w-1],
                stageCost(in, w-1, j-split))
            if cost < f[j][w] {
                f[j][w] = cost
                choice[j][w] = split
            }
        }
    }
}
```

Demo: `go test ./internal/partition/...` — includes a brute-force cross-check on 200 randomised instances.

- $f[j][w]$

Minimal achievable bottleneck when the first j layers are spread over the first w workers. The answer is $f[N][K]$; choice reconstructs the ranges.

- Speed carries the throttle

`Worker.Speed` is an effective multiplier in $(0, 1]$. Thermal derating enters here, so the solver contains no separate thermal branch anywhere.

- Zero-layer stages are legal

That one `if layers == 0` is what lets the DP bypass a badly connected node — exclusion falls out of the objective.

3-4. ACT AND HEAL

From decision to a running pipeline

internal/partition/partition.go -- Decider

```
func (d *Decider) Decide(now time.Time, in Input,
    force bool) (*Result, bool, error) {
    opt, err := Optimal(in)
    if d.current == nil || force ||
        workersChanged(d.current, in.Workers) {
        d.current, d.lastChange = opt, now
        return opt, true, nil // healing path
    }
    currentCost := Evaluate(in, d.current.Splits())
    improved := opt.BottleneckMs <
        currentCost*(1-d.ImprovementFrac)
    if improved && now.Sub(d.lastChange) >= d.Cooldown {
        d.current, d.lastChange = opt, now
        return opt, true, nil // hysteresis passed
    }
}
```

demo -- inject and clear a thermal fault

```
W2=$(docker inspect -f '{{(range .NetworkSettings.Networks)}}\n{{.IPAddress}}{{end}}' kubeedgeinfer-worker2)
curl -X POST http://$W2:9101/gpu/override -d '{"temp_c": 92}'
curl -X POST http://$W2:9101/gpu/override -d '{"clear": true}'
```

Layers migrate off the hot node within roughly 35 s and return once it cools.

• Three exits, one function

A membership change heals immediately; a large enough improvement past the cooldown repartitions; everything else keeps the current split and merely refreshes its cost.

• Evaluate is the honest comparison

The incumbent split is re-scored under current telemetry before comparison — otherwise the threshold would measure against a stale number.

• Applying without a restart

The controller writes the new ranges into the InferencePipeline CRD and pushes them over gRPC with a bumped generation. Workers reject stale-generation forwards; the router refetches the layout and replays the accumulated context. Workers hold no state, so replay is trivially correct.

DEMO

Runbook

1. One command — `./run-demo.sh`

Builds images, creates the kind cluster, deploys everything and opens a live dashboard on `localhost:8080` with the pipeline, per-stage utilisation, eBPF flow sRTTs and a repartition event log.

2. Watch the CRD — `kubectl -n kubeedgeinfer get ipl demo -w`

Assignments and generation update in place as the controller re-plans.

3. Drive load — `kubectl -n kubeedgeinfer port-forward svc/router 8080:8080`

`curl -X POST localhost:8080/generate -d '{"prompt_len":16,"max_new_tokens":8}'`

Requests flow hub-and-spoke through the router across the stages.

4. Inject a fault — `curl -X POST http://$W2:9101/gpu/override -d '{"temp_c": 92}'`

Or use the dashboard buttons. Layers migrate off the hot node, then return after it cools.

5. Reproduce results — `go run ./bench/dpsim`

`python3 bench/run.py --all && python3 bench/plot.py`

`dpsim` reproduces the main deck's slide-11 numbers with no cluster at all; `bench/run.py` needs the live cluster and produces tokens/s, TTFT and bubble time.

Requirements: Linux with BTF at `/sys/kernel/btf/vmlinux`, Docker, kind, kubectl, Python 3.11+.